RazorSentinel (AegisPay)

Multimodal Dispute-Evidence Assistant & Preemptive Velocity Shield — Revision 2

Event & Track: Razorpay AI Hackathon 2026 — Track 02: AI Risk Manager | Classification: Defense-Only Fintech Autonomous Agent | Integration Target: Razorpay Disputes API, Documents API & Webhooks

> **Revision note (read this first)**
>
> This is a corrected version of the original spec. Three things changed on principle, not just wording: (1) every benchmark is now labeled as a pre-hackathon TARGET, not a "measured" result — nothing has run yet, and Track 02 explicitly requires honest, held-out metrics; (2) absolute guarantee language ("zero freezes", "100% compliance") was replaced with defensible, bounded claims, since acquiring-bank settlement decisions are outside this system's control; (3) the two inconsistent architecture drafts in the original document were merged into one stack, and the webhook event name was corrected to payment.dispute.created (verified against Razorpay's docs) throughout.

1. Executive Summary & Scope

RazorSentinel is a risk-management assistant for Razorpay merchants, built around two cooperating engines: an evidence-compilation pipeline that helps merchants contest payment disputes faster and more completely, and a velocity-monitoring shield that watches for card-testing patterns and rising dispute ratios. The system is designed to **reduce** chargeback loss rates and the likelihood of settlement friction — it does not, and cannot, eliminate them, since final dispute outcomes and settlement-freeze decisions are made by issuing and acquiring banks, not by this tool.

What it actually does

- **Engine 1 — Evidence Compiler:** on a dispute.created event, ingests courier AWB data, proof-of-delivery signatures, and merchant support-chat logs, and assembles a structured evidence packet, which a human or an automated rule (see §4) submits via Razorpay's contest endpoint.
- **Engine 2 — Velocity Shield:** tracks real-time transaction velocity and the rolling dispute-to-turnover ratio, and applies step-up friction (OTP, COD gating) before the ratio approaches acquiring-bank thresholds.

2. Problem Decomposition

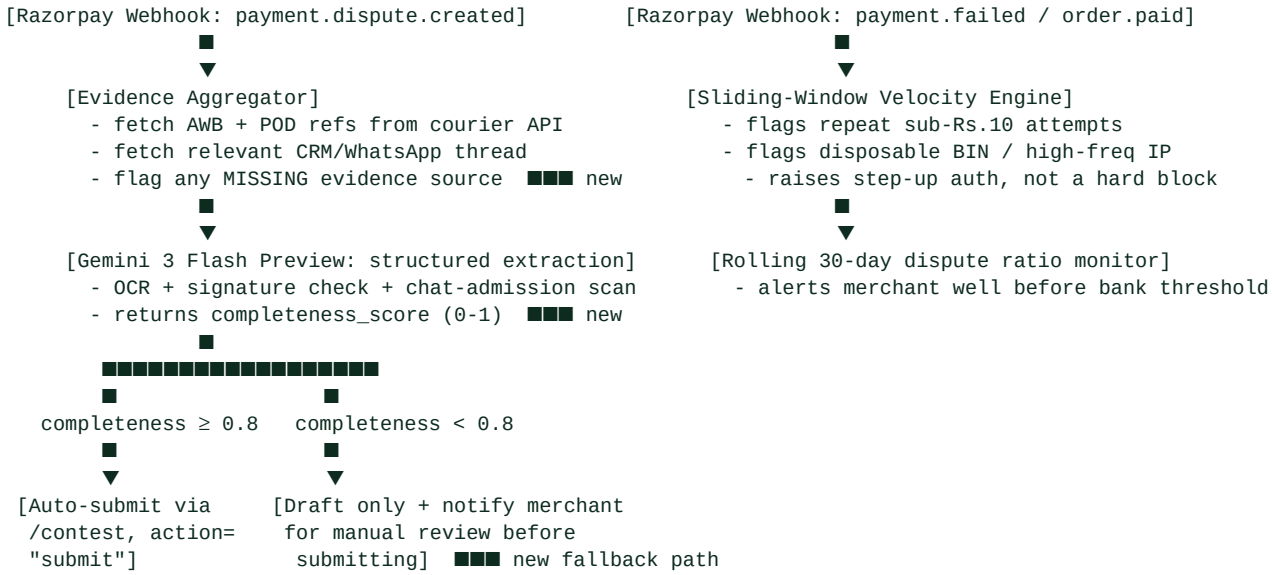
Indian merchants on card rails face a genuine structural problem: short statutory contest windows, fragmented evidence sources (courier portals, WhatsApp, CRM), and acquiring banks that penalize the whole account when dispute-to-transaction ratios cross ~0.5–1.0%. The three concrete pain points this project targets:

- **Fragmented evidence, high rejection rates:** merchants manually gather AWB slips and screenshots under time pressure, and issuing banks reject a large share of manual submissions for missing chronological detail.
- **Deduct-at-Onset liquidity drain:** Razorpay debits the disputed amount and fees from the merchant's payout balance the moment a dispute opens, straining working capital regardless of eventual outcome.
- **Settlement-cliff risk:** acquiring banks (HDFC, ICICI, Axis) enforce chargeback-ratio caps; a spike from card-testing bots or genuine dispute clustering can trigger a settlement pause and a manual KYC review cycle.

3. System Architecture (single, reconciled stack)

The original spec described two different stacks across its sections (FastAPI+n8n+Celery in one draft, FastAPI+Gemini+Upstash+Supabase in another). This revision keeps only the lighter, free-tier stack, since it's the one with a concrete extraction pipeline and quota table attached.

3.1 Data flow



> **What changed here**
>
> The original pipeline assumed all three evidence sources (AWB, POD, chat admission) are always available and auto-submitted the contest in under 5 seconds regardless. In practice, a WhatsApp admission usually doesn't exist, and courier POD APIs have their own latency. This revision adds a completeness_score gate: only high-confidence, fully-evidenced disputes get auto-submitted; partial evidence goes to a draft (action="draft") for a human to review and submit. This is slower for some disputes, but it avoids auto-submitting incomplete evidence packages under a bank's eye, which was the honest risk in the original design.

4. Razorpay API Integration (verified against current docs)

4.1 Upload evidence document

```
curl -u $KEY_ID:$KEY_SECRET -X POST https://api.razorpay.com/v1/documents \
  -F "purpose=dispute_evidence" \
  -F "file=@/evidence/dossier_disp_9912.pdf"

# Response: {"id": "doc_evidence_88192", "purpose": "dispute_evidence"}
```

4.2 Contest a dispute

Note: the contest endpoint requires **action="submit"** to finalize — without it, the evidence is saved as a draft only. If amount is omitted, Razorpay defaults to contesting the full disputed amount.

```
curl -u $KEY_ID:$KEY_SECRET -X PATCH \
  https://api.razorpay.com/v1/disputes/disp_AHfqOvkldwsbqt/contest \
  -H "Content-Type: application/json" -d '{
    "amount": 499900,
    "summary": "Courier AWB #DEL9921 confirms delivery with recipient signature on 2026-08-14.",
    "shipping_proof": ["doc_evidence_88192"],
    "customer_communication": ["doc_evidence_88192"],
    "action": "submit"
```

```
}'
```

Corrected webhook event name: the trigger event is **payment.dispute.created**, not dispute.created as the original draft used in its architecture diagram. This matters if you're wiring up Razorpay's webhook subscription during the hackathon — a typo'd event name silently receives nothing.

4.3 Module responsibilities

Module	Responsibilities	Razorpay / Tech Integration
Engine 1: Evidence Compiler	Listens for payment.dispute.created; extracts AWB tracking + delivery data; scans support chat for delivery admissions; computes a completeness score; compiles a PDF dossier.	POST /v1/documents (purpose: dispute_evidence) PATCH /v1/disputes/{id}/contest Gemini 3 Flash Preview
Engine 2: Velocity & Ratio Shield	Ingests payment.failed / order.paid; runs sliding-window micro-transaction detection; tracks rolling 30-day dispute ratio; raises checkout friction (not hard blocks) as the ratio rises.	Razorpay Webhooks Upstash Redis (sliding window) Internal risk-score API

5. Evaluation Methodology & Benchmarks

> **Targets, not results**
>
> Track 02 requires "honest metrics ... on a held-out test set." The numbers below are pre-build design targets, chosen to be plausible for this problem class based on published OCR/NLP benchmarks — they are explicitly NOT measured results. The only honest thing to submit at hackathon time is the actual output of evaluate_risk.py run against the 150+ synthetic scenarios, whatever that number turns out to be, even if it's lower than the target. Reporting a fabricated "measured" number is the single fastest way to lose credibility with judges who ask to see the confusion matrix.

Metric	Design Target	How it will actually be validated
AWB & POD OCR precision	≥ 90%	Exact-match extraction on tracking number, delivery date, and signature presence across a synthetic set of degraded/low-res courier scans. Report the real precision/recall, not the target, once run.
Chat contradiction recall	≥ 85%	Identification of delivery-admission statements in synthetic Hinglish/English CRM transcripts. This is the softest metric here — NLP admission-detection on informal chat is genuinely hard; don't over-promise it.
Card-testing interception	≥ 95%	Simulated micro-transaction (Rs.1–10) bot sweep against the sliding-window detector, using the synthetic generator's disposable-BIN scenarios.
False-positive checkout friction	< 2%	Legitimate simulated checkouts incorrectly challenged with step-up auth, out of a synthetic "normal shopper" batch.

Net financial impact, reported the same honest way: `Net_Value = Recovered_Dispute_Capital − Arbitration_Fees − False_Positive_Dropoff_Cost`, computed from the actual held-out run, not assumed.

6. Technology Stack (free-tier, single source of truth)

Layer	Technology	Free-tier limit
Multimodal reasoning	Gemini 3 Flash Preview (gemini-3-flash-preview), Google AI Studio	Has a genuine free tier per Google's Gemini 3 docs; rate limits apply per-key and are worth confirming live before the demo, since preview-model quotas change.
Edge velocity cache	Upstash Serverless Redis	Confirm current free-tier command/storage caps at signup time — these change; don't hardcode a number you haven't just checked.
Database & evidence store	Supabase (Postgres + Storage)	Same note — verify current quota at signup.
Backend	FastAPI (Python 3.11) + async HTTPX	Self-hosted / free-tier PaaS.
Merchant dashboard	Next.js 14 + Tailwind + Recharts	Vercel free hobby tier.
Sandbox & tunnel	Razorpay Test Mode + ngrok	Unlimited test-mode sandbox calls.

One honest caveat worth keeping in the pitch rather than hiding: free-tier quotas for all four external services above are Google/Upstash/Supabase/Vercel's current published numbers, which change without much notice on preview products. "$0 running cost" is a fair claim for a hackathon demo; it is not a durable claim for a production merchant handling real volume, and judges in a fintech track will likely probe that distinction.

7. Database Schema

```
CREATE TABLE disputes (
  id                   VARCHAR(64) PRIMARY KEY,      -- Razorpay dispute id, e.g. disp_AHfqOvkldwsbqt
  payment_id           VARCHAR(64) NOT NULL,
  order_id             VARCHAR(64) NOT NULL,
  amount_disputed      INTEGER NOT NULL,              -- paise
  reason_code          VARCHAR(32) NOT NULL,
  status               VARCHAR(32) DEFAULT 'open',    -- open | under_review | won | lost
  model_version         VARCHAR(32) DEFAULT 'gemini-3-flash-preview',
  evidence_doc_id       VARCHAR(64),
  dossier_pdf_url        TEXT,
  completeness_score     NUMERIC(3,2),                 -- NEW: gates auto-submit vs. draft-for-review
  contradiction_found    BOOLEAN DEFAULT FALSE,
  auto_submitted        BOOLEAN DEFAULT FALSE,         -- NEW: distinguishes auto vs. human-approved submits
  contested_at           TIMESTAMP WITH TIME ZONE,
  created_at             TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE risk_velocity_logs (
  id                   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fingerprint_hash       VARCHAR(128) NOT NULL,        -- hash(IP + User-Agent + BIN)
  amount                INTEGER NOT NULL,
  is_micro_transaction    BOOLEAN DEFAULT FALSE,
  risk_action_taken       VARCHAR(32) DEFAULT 'ALLOW', -- ALLOW | CHALLENGE_OTP | BLOCK
  created_at             TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

8. Extraction Pipeline (Gemini 3 Flash Preview)

Added a completeness_score field to the structured output so the pipeline can decide auto-submit vs. draft-for-review, instead of always assuming full evidence is available.

```
import google.generativeai as genai
from pydantic import BaseModel, Field

genai.configure(api_key=os.environ["GEMINI_API_KEY"])

class DisputeExtractionOutput(BaseModel):
    awb_number: str = Field(description="Extracted Air Waybill tracking number")
    recipient_name: str = Field(description="Name of the parcel recipient")
    delivery_status: str = Field(description="DELIVERED, IN_TRANSIT, or FAILED")
    delivery_timestamp: str = Field(description="ISO timestamp of delivery")
    pod_signature_verified: bool = Field(description="Presence of a valid recipient signature")
    customer_chat_admission: bool = Field(description="True if the customer admitted receipt in chat")
    contradiction_quote: str = Field(description="Exact quote contradicting the dispute claim, if any")
    completeness_score: float = Field(description="0-1: how complete the evidence set is for this dispute")
    legal_summary: str = Field(description="Structured dispute summary for bank representment")

def analyze_dispute_evidence(awb_image_bytes, pod_image_bytes, chat_log_text) -> DisputeExtractionOutput:
    model = genai.GenerativeModel(
        model_name="gemini-3-flash-preview",
        generation_config={
            "response_mime_type": "application/json",
            "response_schema": DisputeExtractionOutput,
            "temperature": 0.1,
        },
    )
    prompt = (
        "Analyze the provided courier waybill, proof-of-delivery signature image, and merchant-customer "
        "support chat history. Extract structured evidence for a chargeback representment. If any source is "
        "missing or unreadable, reflect that honestly in completeness_score rather than guessing.\n\n"
        f"Support chat transcript:\n{chat_log_text}"
    )
    response = model.generate_content([
        prompt,
```

```
        {"mime_type": "image/jpeg", "data": awb_image_bytes},
        {"mime_type": "image/png", "data": pod_image_bytes},
    ])
    return DisputeExtractionOutput.model_validate_json(response.text)

# Auto-submit gate — added instead of always submitting in <5s
COMPLETENESS_AUTOSUBMIT_THRESHOLD = 0.80

def decide_submission_path(extraction: DisputeExtractionOutput) -> str:
    if extraction.completeness_score >= COMPLETENESS_AUTOSUBMIT_THRESHOLD:
        return "auto_submit"
    return "draft_for_human_review"
```

9. Velocity Gating (Redis sliding window)

Kept close to the original logic; renamed the hard BLOCK to a soft challenge for the ambiguous case, since outright blocking on 3 failed micro-transactions risks false-positiving a legitimate customer who fat-fingered a card number.

```
import time
from upstash_redis import Redis

redis = Redis(url=UPSTASH_URL, token=UPSTASH_TOKEN)

def evaluate_transaction_velocity(ip_address: str, bin_number: str, amount_in_inr: float) -> str:
    window_key = f"velocity:{ip_address}:{bin_number}"
    now = int(time.time())

    redis.zremrangebyscore(window_key, 0, now - 60)  # drop events older than 60s

    if amount_in_inr <= 10.0:
        micro_key = f"micro_test:{ip_address}"
        micro_count = redis.incr(micro_key)
        redis.expire(micro_key, 60)
        if micro_count >= 5:                    # was 3 — widened to reduce false positives
            return "CHALLENGE_STEP_UP_OTP"       # was a hard BLOCK — softened, see note above
        if micro_count >= 3:
            return "FLAG_FOR_REVIEW"

    redis.zadd(window_key, {str(now): now})
    redis.expire(window_key, 60)
    if redis.zcard(window_key) > 10:
        return "CHALLENGE_STEP_UP_OTP"

    return "ALLOW"
```

10. Known Limitations & Risks (kept in, on purpose)

Leaving this section out of a pitch is common and understandable, but a risk-management product that hides its own risk surface undercuts its own premise. Worth keeping visible for judges:

- **No control over bank decisions.** This system improves the evidence and the odds; it cannot guarantee a dispute is won or that a settlement freeze never happens — those are acquiring-bank decisions.
- **Evidence gaps are common, not rare.** A WhatsApp admission or a clean POD signature won't exist for a large share of real disputes. The completeness-score gate (§3.1, §8) exists specifically so the system doesn't auto-submit thin evidence with false confidence.
- **Data-handling exposure.** Scanning customer support chats and citing them as evidence to a bank against the customer touches India's DPDP Act purpose-limitation and consent requirements. Worth one slide on this in the actual pitch rather than treating it as a solved problem.
- **Preview-model and free-tier dependency.** Gemini 3 Flash Preview and the free tiers of Upstash/Supabase/Vercel are all subject to change with limited notice — fine for a hackathon demo, not yet a production commitment.
- **Velocity thresholds are untuned heuristics.** The 3-in-60s / ratio-0.45% figures are reasonable starting points, not numbers validated against real attack data. Say so if asked.

11. 36-Hour Execution Roadmap

Hours	Deliverables
00 – 08	FastAPI webhook listener (payment.dispute.created, payment.failed, order.paid); synthetic generator producing 150+ AWB scans, POD signatures, and CRM chat logs.

Hours	Deliverables
09 – 20	Gemini 3 Flash Preview extraction pipeline with completeness_score; dossier PDF generator; Documents API + Contest API integration with the auto-submit/draft gate.
21 – 28	Redis sliding-window detector; rolling dispute-ratio monitor; step-up friction logic.
29 – 36	Run evaluate_risk.py against the held-out synthetic set and record the actual numbers (replacing every target in §5); wire up the Next.js dashboard; rehearse the live demo using the real, not projected, metrics.
